

pyside6-webusb v0.0.5a0

vs Chrome WebUSB Compatibility Matrix

Strict technical comparison of the PySide6 / QtWebEngine implementation against the WebUSB model exposed by Chrome / Chromium. English edition.

Scope and interpretation

This matrix distinguishes API shape, observable behavior, browser security requirements, transport behavior, and architecture. A matching method name does not automatically mean browser-level equivalence. Chrome / Chromium is treated as the compatibility reference; pyside6-webusb is a host-application implementation using QtWebEngine, QWebChannel, pyusb/libusb, and optional native acceleration.

1. Status legend

Status	Meaning
Full	Behavior is intentionally implemented to closely match the Chrome/WebUSB model in the supported host environment.
Partial	The API or behavior exists, but there are documented differences, host-environment limits, or incomplete edge cases.
Host-specific	Implemented by pyside6-webusb for desktop/embedded applications and not a browser WebUSB primitive.
Different	The capability exists through a different architecture or semantics; it should not be treated as Chrome-equivalent.
Limited / unverified	Implemented or represented, but physical-hardware coverage or backend behavior is not yet sufficiently verified.
N/A	Not meaningfully comparable because the feature belongs to Chrome's browser process or another browser-only layer.

2. Global architecture

Area	Chrome / Chromium WebUSB	pyside6-webusb	Strict assessment
Web-page API	Browser-provided WebUSB implementation	Injected JavaScript WebUSB polyfill	Partial / Different
Browser bridge	Internal browser implementation	QWebChannel + WebUSBBridge	Different
Native USB layer	Browser/OS USB stack	pyusb/libusb	Different
Device chooser	Browser-controlled chooser	Native device chooser integrated with host bridge	Partial
Permission store	Browser permission model	Origin-aware host-side permission management	Partial / Host-specific
Browser process isolation	Chromium browser architecture	QtWebEngine application architecture	Different
Native acceleration	Not a WebUSB API feature	Optional Rust acceleration	Host-specific
Environment diagnostics	Chrome internal diagnostics / DevTools	pyside6-webusb-doctor and Python report APIs	Host-specific

3. Core WebUSB surface

Feature	Chrome / Chromium	pyside6-webusb	Assessment
navigator.usb	Yes in supported secure contexts	JavaScript polyfill provides navigator.usb	Full API intent / Partial browser semantics
USB object	Yes	Implemented	Full / Partial
USBDevice	Yes	Implemented	Full / Partial
USBConfiguration	Yes	Implemented	Full / Partial
USBInterface	Yes	Implemented	Full / Partial
USBAlternateInterface	Yes	Implemented	Full / Partial
USBEndpoint	Yes	Implemented	Full / Partial
USBConnectionEvent	Yes	Implemented through host/hotplug bridge	Partial
USBInTransferResult	Yes	Implemented	Partial
USBOutTransferResult	Yes	Implemented	Partial
USBIsynchronousInTransferResult	Yes	Implemented with backend limitations	Limited
USBIsynchronousOutTransferResult	Yes	Implemented with backend limitations	Limited
USBIsynchronousInTransferPacket	Yes	Implemented; non-uniform packet cases remain limited	Limited
USBIsynchronousOutTransferPacket	Yes	Implemented; hardware/backend dependent	Limited

4. USBDevice methods and state

Operation	Chrome / Chromium	pyside6-webusb	Assessment
open()	Opens authorized device	Implemented through Python bridge	Partial
close()	Closes device	Implemented	Partial
forget()	Forgets permission for device	Permission management implemented; semantics are host-managed	Partial
selectConfiguration()	Selects configuration	Implemented	Partial
claimInterface()	Claims interface	Implemented	Partial
releaseInterface()	Releases interface	Implemented	Partial
selectAlternateInterface()	Selects alternate interface	Implemented	Partial
controlTransferIn()	Control IN transfer	Implemented	Partial

Operation	Chrome / Chromium	pyside6-webusb	Assessment
controlTransferOut()	Control OUT transfer	Implemented	Partial
clearHalt()	Clears endpoint halt	Implemented	Partial
transferIn()	Bulk/interrupt IN	Implemented	Partial
transferOut()	Bulk/interrupt OUT	Implemented	Partial
isochronousTransferIn()	Isochronous IN	Implemented but not fully hardware-verified	Limited
isochronousTransferOut()	Isochronous OUT	Implemented but backend/hardware dependent	Limited
reset()	Resets USB device	Implemented	Partial
opened/configuration state	Browser tracks WebUSB device state	Bridge/native backend tracks corresponding state	Partial

5. requestDevice() and filters

Feature	Chrome / Chromium	pyside6-webusb	Assessment
requestDevice()	User-mediated chooser required	Native chooser / host bridge	Partial
filters	Standard WebUSB filters	Supported and validated	Full / Partial
exclusionFilters	Supported by WebUSB model	Supported	Partial
vendorId	Supported	Supported	Full
productId	Supported	Supported	Full
serialNumber	Supported where exposed	Supported where backend/device exposes it	Partial
classCode	Supported filter field	Supported	Partial
subclassCode	Supported	Supported	Partial
protocolCode	Supported	Supported	Partial
filter validation	Browser validates WebUSB filter structure	Implementation validates filter structure	Full intent
chooser cancellation	Promise rejection in browser	Mapped through native chooser bridge	Partial
device selection UI ownership	Browser UI	Application/native UI	Different

6. getDevices() and permission semantics

Feature	Chrome / Chromium	pyside6-webusb	Assessment
getDevices()	Returns devices previously granted to origin	Returns already-granted devices for requesting origin	Close / Partial
Origin scoping	Browser-enforced	Origin-aware host-side handling	Partial
Persistent authorization	Browser permission store	Host-side permission state	Different
Permission revocation	Browser UI / permissions APIs	Host-side permission management	Different

Feature	Chrome / Chromium	pyside6-webusb	Assessment
Cross-origin visibility	Blocked by origin permissions	Origin checks intended to prevent leakage	Partial
Frame attribution	Browser knows requesting frame	Frame-aware origin handling	Partial
Arbitrary system device enumeration	Not exposed to page	Not exposed as unrestricted page API	Close
Host pre-authorization	Not a normal page API	grant_device_for_origin() available to host application	Host-specific

7. Secure context and Permissions Policy

Requirement	Chrome / Chromium	pyside6-webusb	Assessment
Secure Context	WebUSB requires secure context	QtWebEngine application controls navigation/origin environment; not identical to Chrome browser policy	Partial / Different
HTTPS origin	Standard browser security boundary	Supported as an origin input	Partial
Permissions Policy: usb	Browser-enforced policy applies	Must be enforced/controlled by the QtWebEngine host environment; not equivalent to Chrome browser policy enforcement	Partial / Host-dependent
iframe policy inheritance	Browser-defined	Frame-aware handling exists, but not identical to Chromium's browser process policy stack	Partial
Opaque origins	Browser-defined handling	Host/polyfill behavior can differ	Limited
User activation	Browser activation model	Host/polyfill must emulate or enforce where applicable	Partial
Browser UI permission prompt	Chrome-controlled	Application/native chooser	Different

8. Security restrictions

Security control	Chrome / Chromium	pyside6-webusb	Assessment
Protected USB interface classes	Blocks protected classes	Protected-interface checks implemented	Close / Partial
Security-key blocklist	Chromium security-key blocklist	Chromium-derived security-key blocklist	Close
Transfer validation	Browser validates transfer parameters	Bridge validates transfer parameters	Close / Partial
Transfer-size safety	Chrome has browser-specific limits	32 MiB treated as warning; separate host-side safety ceiling	Different by design
Host resource limits	Browser process/resource controls	Host-side limits	Different
Origin authorization	Browser permission system	Origin-aware permission manager	Partial
Direct native access from page	Not possible	Web page cannot directly call pyusb/libusb; bridge mediates access	Close
Bridge bypass resistance	Browser process boundary	QWebChannel/bridge validation is required	Partial / Host-dependent
Malicious descriptors	Browser USB stack validates behavior	Native/backend validation and hardening	Partial
Resource exhaustion	Browser has internal safeguards	Host-side safety limits and hardening	Partial

9. Transfer behavior

Transfer area	Chrome / Chromium	pyside6-webusb	Assessment
Control transfers	Supported	Supported	Partial
Bulk transfers	Supported	Supported	Partial
Interrupt transfers	Supported	Supported	Partial
Isochronous transfers	Supported	Supported with limitations	Limited
stall handling	WebUSB result/status mapping	Mapped where possible	Partial
babble handling	Browser/backend-defined WebUSB result	Mapped where possible	Partial
DataView / ArrayBuffer model	Web API data types	Polyfill maps to compatible JS objects	Partial
32 MiB reference	Important Chromium implementation reference	Warning above 32 MiB rather than unconditional rejection	Different
Host ceiling	Browser-internal	Independent host-side safety ceiling	Host-specific
Non-uniform isochronous packets	Supported by browser implementation where backend permits	Still limited	Limited

10. Hotplug and events

Feature	Chrome / Chromium	pyside6-webusb	Assessment
device connect event	Supported	Hotplug monitoring and bridge event path	Partial
device disconnect event	Supported	Hotplug monitoring and bridge event path	Partial
Origin filtering of events	Browser permission/origin model	Origin-aware event delivery intended	Partial
Cross-origin hotplug leakage prevention	Browser-enforced	Explicit hardening/tests target this case	Partial
Native watcher	Browser USB service/backend	pyusb/libusb/backend	Different
Timing guarantees	Browser implementation-defined	Backend/OS dependent	Different

11. Error and exception behavior

Area	Chrome / Chromium	pyside6-webusb	Assessment
DOMException-style API errors	Standard browser exceptions	Polyfill maps native failures to JS/WebUSB-style errors	Partial
NotFoundError	Used for missing/cancelled selection cases	Chooser/backend errors mapped where applicable	Partial
SecurityError	Browser security boundary	Host/polyfill security checks	Partial
NetworkError	Browser/backend failures	Bridge/backend failures may be mapped	Partial
InvalidStateError	State-machine errors	Bridge tracks device state	Partial
OperationError	Transfer/backend operation failures	Backend failures mapped where possible	Partial

Area	Chrome / Chromium	pyside6-webusb	Assessment
Exact message compatibility	Not guaranteed across browser versions	Not intended as byte-for-byte Chrome clone	No
Error timing	Browser-defined	Python/Qt/backend timing can differ	Different

12. DevTools and diagnostics

Feature	Chrome / Chromium	pyside6-webusb	Assessment
DevTools inspection	Chrome DevTools	QtWebEngine DevTools / F12	Close for inspection
Implementation namespace	Chrome internal objects are not public API	window.__pysideWebUSB	Host-specific
Granted-device diagnostics	Browser permission UI/internal state	Debug namespace exposes relevant state	Host-specific
Transfer-limit diagnostics	Browser-specific internals	Debug namespace reports limits/explanations	Host-specific
Environment report	Not a WebUSB page API	environment_report()/format_environment_report()	Host-specific
Doctor CLI	No equivalent public WebUSB API	pyside6-webusb-doctor	Host-specific

13. Host-only capabilities

Capability	Chrome WebUSB	pyside6-webusb	Assessment
Host pre-authorization	Not exposed as normal page API	grant_device_for_origin()	Extra
Python bridge	No	Yes	Extra
QWebChannel integration	No	Yes	Extra
pyusb/libusb direct backend	Browser-internal implementation differs	Yes	Different / Extra
Rust acceleration	Not a WebUSB API	Optional	Extra
Environment diagnostics	Not a WebUSB API	Yes	Extra
Application-controlled chooser	Browser chooser is separate	Host application controls native chooser integration	Different
Embedded deployment	Chrome is a browser	Designed for PySide6 embedded applications	Different

14. TypeScript and API surface

Area	Chrome / Chromium	pyside6-webusb	Assessment
Web IDL/API shape	Defined by WebUSB specification	TypeScript definitions and JS polyfill	Close / Partial
USBDevice typing	Native browser IDL	Provided TypeScript definitions	Close
USBConfiguration typing	Native browser IDL	Provided	Close
USBInterface typing	Native browser IDL	Provided	Close

Area	Chrome / Chromium	pyside6-webusb	Assessment
USBEndpoint typing	Native browser IDL	Provided	Close
Runtime identity semantics	Native browser objects	Polyfill/bridge objects	Different
Prototype identity / instanceof	Browser-native Web IDL objects	Polyfill-dependent	Potential difference
Byte-for-byte Chrome behavior	Not applicable as a compatibility guarantee	Explicitly not a goal	No

15. Overall compatibility judgment

Dimension	Assessment	Reason
API surface	High	Major WebUSB objects and operations are implemented.
Basic device selection	High / Partial	Native chooser replaces Chrome browser UI.
Permission model	Medium-High	Origin-aware and granted-device model exists, but storage/enforcement is host-side.
Security model	Medium-High	Multiple protections exist, but the browser-process security boundary is different.
Transfer compatibility	High for common transfers	Control/Bulk/Interrupt are implemented; backend behavior can differ.
Isochronous compatibility	Limited	Physical-hardware and packet-length edge cases remain experimental.
Chrome implementation equivalence	Low by definition	Architecture is fundamentally different.
WebUSB application portability	Potentially high	Applications using standard API patterns can be compatible, but browser assumptions may fail.
Production readiness	Experimental	v0.0.5a0 remains an experimental release.

16. Important compatibility conclusions

1. pyside6-webusb should be described as WebUSB-compatible, not as a Chrome WebUSB clone. Its API surface is deliberately modeled after WebUSB, while the implementation is a Qt/Python/native bridge.
2. The strongest compatibility area is the ordinary WebUSB programming model: navigator.usb, device objects, filters, permissions concepts, and common Control/Bulk/Interrupt transfers.
3. The largest differences are browser security architecture, permission storage, native chooser ownership, Permissions Policy enforcement, user-activation semantics, exact DOMException behavior, and backend-specific USB behavior.
4. Isochronous transfers require the most caution. The implementation exists, but physical hardware and non-uniform packet-length cases remain less mature than ordinary transfers.
5. The 32 MiB transfer behavior is intentionally not identical to Chrome: pyside6-webusb warns above 32 MiB and applies a separate host-side safety ceiling. This is a deliberate compatibility/security trade-off.
6. Host-only functions such as pre-authorization and environment diagnostics are additional capabilities, not WebUSB compatibility features.

Reference basis

This document uses the WebUSB API model and Chrome/Chromium behavior as the comparison baseline, while treating the pyside6-webusb v0.0.5a0 implementation as the implementation under test. Because browser internals and QtWebEngine/libusb behavior vary by version and operating system, this matrix intentionally avoids claiming byte-for-byte or error-message-for-error-message equivalence.